



Semester Project — Deliverable 2 (Backend)
Smart University Timetable Management System

BS Software Engineering (Blue 6th)

Roll No: 086

Name: Muhammad Basit

TOTAL GRADE POINT: 10

GRADE POINT ALLOCATED:

ASSESSOR'S COMMENTS

Instructor: Museb Khalid

Abstract

Universities often face challenges in managing class timetables and handling lecture rescheduling requests manually. Students usually communicate with teachers through phone calls or social media, which may cause delays and confusion. The Smart University Timetable Management System is a web-based application developed to simplify timetable management by providing a centralized platform for students, teachers, and advisors. The system allows students to view their personal timetable, access the complete timetable, check available free slots, and submit lecture rescheduling requests. Teachers can review these requests and either approve or reject them, while advisors can monitor all activities within the system. The frontend is developed using Vue.js, whereas the backend uses Node.js, Express.js, MongoDB, and Mongoose. JWT authentication is used to provide secure role-based access for all users.

1. Introduction:

Timetable management is an important part of every educational institution. In many universities, timetable changes are handled manually through verbal communication or social media groups. This process is time-consuming and often causes confusion among students and teachers.

The Smart University Timetable Management System provides a digital solution where students can request timetable changes online. Teachers can review these requests and approve or reject them. Advisors can monitor all requests and activities from a single dashboard. This system improves communication and reduces manual work.

2. System Overview:

The Smart University Timetable Management System follows a three-tier architecture consisting of the presentation layer, application layer, and database layer.

The presentation layer is developed using Vue.js and provides interactive dashboards for students, teachers, and advisors. The application layer is built using Express.js and Node.js, which process user requests, perform authentication, and communicate with the database. MongoDB stores all system data including users, subjects, semesters, sections, timetable slots, notifications, and reschedule requests.

The architecture ensures proper separation between the frontend, backend, and database, making the application easier to maintain and extend.

Layer	Technology	Purpose
Presentation Layer	Vue.js, HTML, CSS, JavaScript	Provides the user interface and dashboards
Application Layer	Node.js, Express.js	Handles business logic and REST APIs
Database Layer	MongoDB, Mongoose	Stores all application data securely

3. Technologies Used:

The application is developed using modern web technologies. Vue.js is used to create a responsive single-page application. Express.js provides REST APIs that connect the frontend with MongoDB. Mongoose simplifies database operations by defining schemas and models. JWT provides secure authentication and authorization based on user roles.

Technology	Purpose
Vue.js 3	Frontend JavaScript Framework
Vite	Development Server and Build Tool
Vue Router	Single Page Application Routing
Axios	HTTP Requests between Frontend and Backend
HTML5	Structure of Web Pages
CSS3	User Interface Styling
JavaScript (ES6)	Frontend and Backend Programming
Node.js	Server-side JavaScript Runtime
Express.js	REST API Development
MongoDB	NoSQL Database
Mongoose	MongoDB Object Modeling
JWT	User Authentication
dotenv	Environment Variable Management
CORS	Cross-Origin Resource Sharing
Nodemon	Automatic Server Restart During Development

4. System Architecture:

The system follows a client-server architecture.

1. User opens the Vue.js application.
2. User logs into the system.
3. Backend verifies user credentials.
4. JWT token is generated.
5. User accesses features according to their role.
6. Backend communicates with MongoDB to retrieve or update data.
7. Response is displayed on the frontend.

5. System Modules:

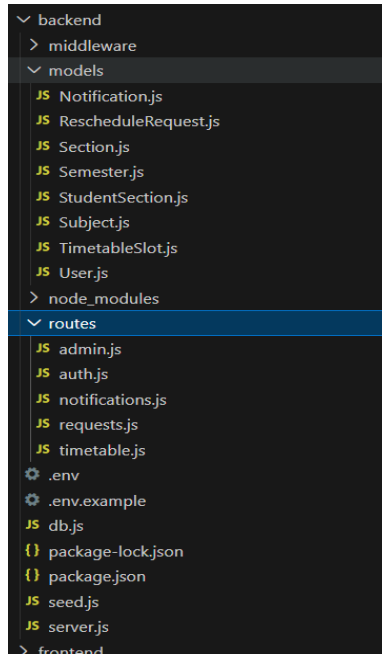
Module	Description
Authentication Module	Handles user registration, login, JWT authentication, and role-based access for students, teachers, and advisors.
Student Module	Allows students to view timetables, find free slots, submit reschedule requests, and receive notifications.
Teacher Module	Enables teachers to review pending requests, approve or reject them, and view request history.
Advisor Module	Allows advisors to monitor all timetable requests and system activities.
Timetable Management Module	Displays the complete timetable and individual student timetable.
Free Slot Management Module	Identifies available timetable slots that can be used for lecture rescheduling.
Reschedule Request Module	Manages the creation, tracking, approval, and rejection of timetable change requests.
Notification Module	Sends notifications to users whenever the request status changes.
Database Module	Stores users, subjects, sections, timetables, requests, and notifications securely in MongoDB.

6. Backend Architecture:

A. Project Structure:

The backend is developed using **Node.js** and **Express.js**.

The project is organized into different folders including routes, models, middleware, configuration files, and the database connection. Each file has a specific responsibility, making the application modular and easy to maintain.



B. Database Schema:

MongoDB stores all project data using Mongoose schemas. Every collection represents a different part of the system such as users, semesters, timetable slots, notifications, and requests.

Collection	Description
Users	Stores student, teacher, and advisor information.
Semesters	Stores semester details.
Sections	Stores section information.
Subjects	Stores subject information.
StudentSections	Maps students to their assigned sections.
TimetableSlots	Stores lecture schedules for all sections.
RescheduleRequests	Stores timetable change requests submitted by students.
Notifications	Stores notifications generated after request approval or rejection.

User Collection Fields

Field	Type	Description
name	String	User's full name
email	String	User login email
password	String	Encrypted password
role	String	Student, Teacher, or Advisor

Timetable Slot Fields

Field	Type	Description
section_id	ObjectId	Section assigned to lecture
subject_id	ObjectId	Subject information
teacher_id	ObjectId	Teacher conducting lecture
day	String	Lecture day
start_time	String	Lecture start time
end_time	String	Lecture ending time
room	String	Classroom number

Reschedule Request Fields

Field	Type	Description
student_id	ObjectId	Student submitting request
slot_id	ObjectId	Original timetable slot
requested_day	String	Requested lecture day
requested_start	String	Requested start time
requested_end	String	Requested ending time
requested_room	String	Requested classroom
reason	String	Reason for rescheduling
status	String	Pending, Approved, Rejected

C. Rest API Endpoints:

Method	Endpoint	Description
POST	/api/auth/register	Register a new user
POST	/api/auth/login	User login
GET	/api/timetable/my	Display student's timetable
GET	/api/timetable/all	Display complete timetable
GET	/api/timetable/free-slots	Display available free slots
POST	/api/requests	Submit timetable reschedule request
GET	/api/requests/pending	View teacher's pending requests
GET	/api/requests/history	View teacher request history
POST	/api/requests/:id/approve	Approve request
POST	/api/requests/:id/reject	Reject request
GET	/api/requests/all	Advisor views all requests
GET	/api/notifications	Retrieve user notifications

7. Data Flow:

A. Login Flow:

The user opens the application and enters their email and password on the login page. The frontend sends a POST request to the backend authentication API. The backend verifies the user credentials using MongoDB. If the credentials are correct, a JSON Web Token (JWT) is generated and returned to the frontend. The token is stored by the application and used for accessing protected routes. According to the user's role (Student, Teacher, or Advisor), the appropriate dashboard is displayed.

B. Viewing Timetable

After logging in, a student can access the **My Timetable** page. The frontend sends a request to the backend, which retrieves timetable information from the **TimetableSlots** collection. The timetable is returned to the frontend and displayed in a structured table. Users can also view the complete timetable of all sections.

C. Finding Free Slots

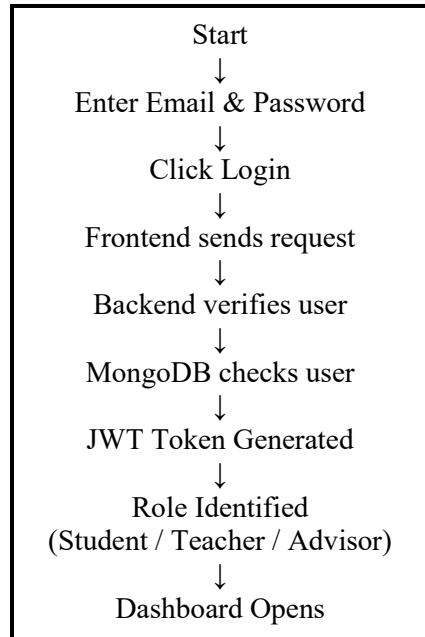
Students can open the **Free Slots** page to check available lecture periods. The backend compares the existing timetable schedule and identifies empty time slots. These available slots are returned to the frontend, allowing students to choose a suitable time when submitting a reschedule request.

D. Reschedule Request Flow

A student selects a lecture from the timetable and submits a reschedule request by providing the requested day, time, classroom, and reason. The frontend sends the request to the backend using the **POST /api/requests** endpoint. The backend validates the request, checks for timetable conflicts, and stores it in the **RescheduleRequests** collection with a status of **Pending**.

E. Approval Flow

Teachers can view all pending requests from their dashboard. After reviewing a request, the teacher can approve or reject it. The backend updates the request status in MongoDB and creates a notification for the student. The student can then view the updated request status from the notification page.



8. Security Considerations:

The Smart University Timetable Management System implements several security measures to protect user data and ensure authorized access.

- **JWT Authentication:** Users must log in before accessing protected pages.
- **Role-Based Authorization:** Students, teachers, and advisors can only access features assigned to their roles.
- **Environment Variables:** Sensitive information such as database connection strings and JWT secrets are stored in the .env file.
- **Input Validation:** Required fields are validated before processing requests.
- **Password Encryption:** User passwords are encrypted before being stored in MongoDB using the bcrypt library.
- **Protected API Routes:** All secure routes verify the JWT token before executing any operation.

9. Conclusion:

The **Smart University Timetable Management System** successfully provides a digital solution for managing university timetables and lecture rescheduling requests. The system simplifies communication between students, teachers, and advisors by replacing manual processes with an online platform.

Students can view their timetable, find available free slots, and submit reschedule requests, while teachers can approve or reject requests efficiently. Advisors can monitor all system activities through a centralized dashboard.

The project demonstrates the practical implementation of modern web development technologies including **Vue.js**, **Node.js**, **Express.js**, **MongoDB**, **Mongoose**, and **JWT Authentication**. The modular architecture, secure authentication mechanism, and role-based access control make the application reliable, maintainable, and scalable. Overall, the system improves timetable management, reduces communication delays, and provides a better user experience for educational institutions.

REFERENCES:

- [1] Vue.js Team, *Vue.js Documentation*, 2026. Available: <https://vuejs.org/>
- [2] Node.js Foundation, *Node.js Documentation*, 2026. Available: <https://nodejs.org/>
- [3] Express.js, *Express Documentation*, 2026. Available: <https://expressjs.com/>
- [4] MongoDB Inc., *MongoDB Documentation*, 2026. Available: <https://www.mongodb.com/docs/>
- [5] Mongoose Team, *Mongoose Documentation*, 2026. Available: <https://mongoosejs.com/docs/>
- [6] JWT.io, *JSON Web Token Introduction*, 2026. Available: <https://jwt.io/>
- [7] Axios Team, *Axios Documentation*, 2026. Available: <https://axios-http.com/>
- [8] MDN Web Docs, *JavaScript Guide*, 2026. Available: <https://developer.mozilla.org/>
- [9] Vite Team, *Vite Documentation*, 2026. Available: <https://vitejs.dev/>
- [10] HTML Living Standard, *WHATWG HTML Documentation*, 2026. Available: <https://html.spec.whatwg.org/>